



Forge Kernel Testing & Benchmarking Rules



Testing guidelines that **all Forge team members** must follow while benchmarking kernels.

Rule 1: Benchmark Both in Isolation and with a Model

Every kernel must be benchmarked in **two** settings:



a) In Isolation — Run the standalone kernel on dummy data to measure raw kernel performance.



b) With a Model — Swap an actual HuggingFace model's kernels with our written kernel. This is primarily to ensure a fair comparison with Unsloth, since they fuse different consecutive layers.

▼ Example: Why model-level benchmarking matters

Say an Attention Module is: **RMSNorm** → **Attention** → **MLP** → **Residual Add** (pseudo example).

- **Liger** would optimise the RMSNorm layer in isolation.
- **Unsloth** would fuse the forward and backward pass for the entire chain: RMSNorm → Attention → MLP → Residual Add.

So it would be difficult to get numbers for just RMSNorm in that fused chain. How exactly to compare our kernels with Unsloth's fused kernels is an open problem — we'll figure it out as we go.

Rule 2: Use Free Colab TPUs Before Spinning Up A100s

Get isolated kernels **tested, working, and benchmarked on Google Colab free TPUs first**, before benchmarking on RunPod's A100.



Why? Cost saving — even starting RunPod spins up A100s which starts billing immediately.

Rule 3: Run 8–10 Iterations and Report Statistics

Test kernels **at least 8–10 times** and average to get actual numbers.



The first few runs often cause **cache warmup and cold start** — when we first launch a kernel, the chips are literally *cold*.

Report the following statistics across all runs:

Metric	Why It Matters
Mean	Average performance across runs
Median	Robust central tendency (less sensitive to outliers)

Metric	Why It Matters
Std Dev	Variance / consistency of results
P95	Tail latency — a kernel fast only 90% of the time still ruins user experience 10% of the time
P99	Worst-case latency for reliability guarantees

Rule 4: Never Use `time.time()` — Use CUDA Synchronization

Triton calls are **asynchronous**. Using `time.time()` will give you wrong numbers.



Always use `torch.cuda.synchronize()` or **CUDA events** to measure kernel time.

Rule 5: Flush L2 Cache Between Runs

Remember to **flush the L2 cache between runs** to avoid inflated performance from cached data.

Rule 6: Scaling Analysis — Sweep Across Dimensions

A kernel might be fast for sequence length 1024 but slow for 8192 (Shaurya's kernel was the reverse ~~100~~).

You must have a sweep table across:

- **Sequence length** (e.g., 512, 1024, 2048, 4096, 8192)
- **Batch size** (e.g., 1, 2, 4, 8, 16)
- **Dimension size** (e.g., hidden dims relevant to the kernel)

Rule 7: Performance Profiling — Required Metrics

The following metrics must be included **at the very least** in every benchmark report.

7a) Arithmetic Intensity

The ratio of floating-point operations (FLOPs) to data movement (Bytes).



Formula: $AI = \frac{\text{Total FLOPs}}{\text{Total Bytes Accessed (Read + Write)}}$

Why: It tells you if your kernel is **Compute-bound** (limited by the GPU's cores) or **Memory-bound** (limited by VRAM speed). Most kernels like RMSNorm or Softmax are heavily memory-bound.

7b) Theoretical Peak Utilisation

Don't just report "10ms." Report **how much of the hardware you are actually using**.

Bound Type	Formula
Compute	(Actual TFLOPS / Peak TFLOPS of GPU) × 100
Memory	(Measured GB/s / Peak BW of GPU) × 100



If your kernel is hitting **< 40% of peak**, there is likely a major optimization (like memory coalescing or shared memory usage) left on the table.

7c) Numerical Correctness (rtol / atol)

Speed is zero without accuracy. You must report the maximum difference against `torch.nn` equivalents.

Tolerance	Definition	FP16/BF16 Target	FP32 Target
<code>atol</code> (Absolute)	Maximum allowed difference: $ a - b $	1e-3	1e-5
<code>rtol</code> (Relative)	Difference relative to magnitude: $\frac{ a-b }{ b }$	1e-3	1e-5

7d) The Roofline Model (Ridge Point Graph)

This is the **gold standard** for high-performance computing. It plots performance (TFLOPS) against Arithmetic Intensity (FLOPs/Byte).

Region	What It Represents	Implication
Slanted Line	The Memory Wall	Kernel is bottlenecked by memory bandwidth
Horizontal Plateau	The Compute Ceiling	Kernel is bottlenecked by the GPU's raw math power
Ridge Point	The Intersection	The specific intensity required to fully saturate the GPU

Rule 8: Comparing Against C++/CUDA Kernels

Some competitors (Unsloth, Flash Attention) ship C++/CUDA kernels, not Triton. To benchmark against these, use **PyTorch C++ Extensions** (`torch.utils.cpp_extension.load`) to JIT-compile the `.cu` file and load it as a Python-callable module. This gives you a function that takes and returns `torch.Tensor` directly — so you can plug it into the **exact same benchmarking harness** (CUDA events, L2 flush, same `benchmark_kernel()` function) with zero-copy tensor sharing and no data-transfer overhead polluting the numbers.

- ▼ How to load and benchmark a C++/CUDA kernel step-by-step

Step 1: Write a thin C++ wrapper that binds your CUDA kernel to Python via pybind11 (PyTorch handles pybind automatically).

```
// rmsnorm_wrapper.cpp
#include <torch/extension.h>

// Forward declaration — implemented in rmsnorm.cu
torch::Tensor rmsnorm_fwd_cuda(torch::Tensor x, torch::Tensor weight, float eps);
torch::Tensor rmsnorm_bwd_cuda(torch::Tensor grad_out, torch::Tensor x, torch::Tensor weight, float eps);

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("forward", &rmsnorm_fwd_cuda, "RMSNorm forward (CUDA)");
    m.def("backward", &rmsnorm_bwd_cuda, "RMSNorm backward (CUDA)");
}
```

Step 2: JIT-compile and load from Python — no manual `nvcc` needed.

```
from torch.utils.cpp_extension import load

cuda_rmsnorm = load(
    name="cuda_rmsnorm",
    sources=["kernels/rmsnorm.cu", "kernels/rmsnorm_wrapper.cpp"],
    extra_cuda_cflags=["-O3", "--use_fast_math"],
    verbose=True, # shows nvcc compilation output
)

# Now benchmark it identically to Triton kernels
stats = benchmark_kernel(cuda_rmsnorm.forward, x, weight, eps)
```

Step 3: (Optional) Wrap in `torch.autograd.Function` if you need backward pass comparison.

```
class CUDARMSNorm(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight, eps):
        output = cuda_rmsnorm.forward(x, weight, eps)
        ctx.save_for_backward(x, weight)
        ctx.eps = eps
        return output

    @staticmethod
    def backward(ctx, grad_output):
        x, weight = ctx.saved_tensors
        grad_input = cuda_rmsnorm.backward(grad_output, x, weight, ctx.eps)
        return grad_input, None, None
```

Step 4: Add a "CUDA C++" column in your benchmark tables alongside PyTorch / Liger / Unsloth / Forge. The numbers are directly comparable since they share the same tensors, timing method, and measurement harness.

Other approaches (less preferred):

- **CuPy `RawKernel`** — loads CUDA source strings directly in Python. Quick for one-off comparisons but hard to manage multi-file kernels.
- **Standalone binary + subprocess** — compile with `nvcc`, run separately. Loses the apples-to-apples guarantee (different memory allocator, different launch overhead). Only use if the CUDA kernel absolutely cannot be adapted to take torch tensors.

Ideal Benchmark Report

★ **This is the gold-standard template.** Every kernel benchmark report you submit must follow this structure. The example below uses **RMSNorm** on an **A100 80GB SXM** to show exactly what is expected.

0. Test Environment (always state this first)

GPU	NVIDIA A100 80GB SXM
Peak BF16 TFLOPS	312 TFLOPS
Peak Memory BW	2,039 GB/s
Ridge Point	~153 FLOPs/Byte
CUDA Version	12.1
PyTorch Version	2.3.0
Triton Version	2.3.0
Kernel Under Test	RMSNorm (Forward + Backward)
Reference Model	Qwen3-8B (hidden=4096)
Dtype	bf16 (fp32 accumulation internally)
Warmup Runs	3 (discarded)
Timed Runs	10
L2 Cache	Flushed between every run

Timing Method	torch.cuda.Event (NOT time.time)
---------------	----------------------------------

1. Measurement Code (reference snippet)



Every benchmark must use this pattern. Copy and adapt it for your kernel.

```
import torch

def benchmark_kernel(fn, *args, warmup=3, runs=10, flush_l2=True):
    """Standard Forge benchmarking harness."""
    # Warmup (discarded)
    for _ in range(warmup):
        fn(*args)
        torch.cuda.synchronize()

    # L2 flush buffer (32MB > L2 cache size)
    flush_buf = torch.empty(8 * 1024 * 1024, dtype=torch.float32, device='cuda')

    start_events = [torch.cuda.Event(enable_timing=True) for _ in range(runs)]
    end_events = [torch.cuda.Event(enable_timing=True) for _ in range(runs)]

    for i in range(runs):
        if flush_l2:
            flush_buf.zero_() # flush L2 cache
            torch.cuda.synchronize()
        start_events[i].record()
        fn(*args)
        end_events[i].record()

    torch.cuda.synchronize()
    times = [s.elapsed_time(e) * 1000 for s, e in zip(start_events, end_events)] #
    μs

    times_t = torch.tensor(times)
    return {
        'mean': times_t.mean().item(),
        'median': times_t.median().item(),
        'std': times_t.std().item(),
        'min': times_t.min().item(),
        'max': times_t.max().item(),
        'p50': times_t.quantile(0.50).item(),
        'p95': times_t.quantile(0.95).item(),
        'p99': times_t.quantile(0.99).item(),
    }
```

Part A — Isolation Benchmarks



Isolation = standalone kernel on synthetic tensors. This measures raw kernel speed without model overhead.

A1. Forward Pass Latency — Sequence Length Sweep

Fixed: batch=4, hidden=4096, dtype=bf16

seq_len	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)	Forge Speedup vs PyTorch
512	45.2	19.3	18.6	18.9	2.39x
1024	82.7	35.8	34.2	34.8	2.38x
2048	158.4	68.1	65.4	66.3	2.39x
4096	312.6	134.7	129.3	131.2	2.38x
8192	621.8	267.4	256.9	260.5	2.39x

A2. Backward Pass Latency — Sequence Length Sweep

Fixed: batch=4, hidden=4096, dtype=bf16

seq_len	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)	Forge Speedup vs PyTorch
512	71.4	31.2	29.8	30.4	2.35x
1024	136.8	58.7	56.1	57.2	2.39x
2048	268.3	114.6	109.8	111.7	2.40x
4096	531.9	227.4	218.1	221.6	2.40x
8192	1058.2	452.1	434.2	441.3	2.40x

A3. Forward + Backward Combined — Sequence Length Sweep

Fixed: batch=4, hidden=4096, dtype=bf16

seq_len	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)	Forge Speedup vs PyTorch
512	116.6	50.5	48.4	49.3	2.37x
1024	219.5	94.5	90.3	92.0	2.39x
2048	426.7	182.7	175.2	178.0	2.40x
4096	844.5	362.1	347.4	352.8	2.39x
8192	1680.0	719.5	691.1	701.8	2.39x

A4. Detailed Run Statistics — Reference Configuration



Config: batch=4, seq_len=2048, hidden=4096, bf16, Forward+Backward, 10 timed runs (3 warmup discarded)

Statistic	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)
Mean	426.7	182.7	175.2	178.0
Median	425.1	182.2	174.8	177.4
P50	425.1	182.2	174.8	177.4
P95	433.8	186.4	178.9	181.6
P99	437.2	188.1	180.3	183.2
Std Dev	4.3	1.9	1.7	1.8

Statistic	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)
Min	421.4	179.8	172.6	175.1
Max	438.6	189.3	181.4	184.1



How to read this: Mean $\approx$ Median means the distribution is symmetric (good). Low Std Dev means the kernel is consistent. If P99 $\gg$ Mean, you have outlier spikes — investigate.

A5. Scaling Analysis — Batch Size Sweep

Fixed: *seq_len=2048, hidden=4096, dtype=bf16, Fwd+Bwd*

batch	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)	Forge Speedup
1	112.4	48.1	46.2	46.9	2.40x
2	218.6	93.7	89.9	91.3	2.39x
4	426.7	182.7	175.2	178.0	2.40x
8	848.2	363.4	348.7	354.1	2.40x
16	1689.7	723.8	694.6	705.2	2.40x

A6. Scaling Analysis — Hidden Dimension Sweep

Fixed: *batch=4, seq_len=2048, dtype=bf16, Fwd+Bwd*

hidden_dim	PyTorch (μs)	Liger (μs)	Unsloth (μs)	Forge (μs)	Forge Speedup
1024	112.8	49.2	47.1	47.9	2.35x
2048	218.3	94.1	90.4	91.8	2.38x
4096	426.7	182.7	175.2	178.0	2.40x
8192	847.1	361.8	347.2	352.4	2.40x
11008	1138.6	486.2	466.8	473.9	2.40x

A7. Peak VRAM Usage

Fixed: *batch=4, hidden=4096, dtype=bf16, Fwd+Bwd*

seq_len	PyTorch (MB)	Liger (MB)	Unsloth (MB)	Forge (MB)	Forge VRAM Saved vs PyTorch
512	96.0	48.0	32.0	40.0	58.3%
1024	192.0	96.0	64.0	80.0	58.3%
2048	384.0	192.0	128.0	160.0	58.3%
4096	768.0	384.0	256.0	320.0	58.3%
8192	1536.0	768.0	512.0	640.0	58.3%



Note: Unsloth uses less VRAM here because they avoid saving intermediate tensors by fusing the entire layer backward pass. Forge's per-kernel autograd stores `ctx.save_for_backward` tensors individually — this is the modularity vs memory tradeoff.

A8. Performance Profiling

Arithmetic Intensity

Pass	Total FLOPs	Total Bytes (R+W)	Arithmetic Intensity (FLOPs/Byte)	A100 Ridge Point	Classification
Forward	~168M	~134 MB	~1.3	~153	Memory-Bound
Backward	~302M	~147 MB	~2.1	~153	Memory-Bound

Config: batch=4, seq_len=2048, hidden=4096

Theoretical Peak Utilisation

Framework	Achieved TFLOPS	Compute Util (%)	Measured BW (GB/s)	Memory BW Util (%)	Bottleneck
PyTorch	1.1	0.4%	738	36.2%	Memory
Liger	2.6	0.8%	1,558	76.4%	Memory
Unsloth	2.7	0.9%	1,623	79.6%	Memory
Forge	2.7	0.9%	1,596	78.3%	Memory



Forge hits 78.3% of peak memory bandwidth — well above the 40% threshold. RMSNorm is memory-bound, so memory BW utilisation is the metric that matters, not compute utilisation.

Numerical Correctness

Tested against `torch.nn.RMSNorm` reference output

Framework	max atol (bf16)	max rtol (bf16)	max atol (fp32)	max rtol (fp32)	gradcheck (fp64)	Verdict
Liger	3.1e-4	2.7e-4	1.2e-6	8.5e-7	PASS	PASS
Unsloth	4.3e-4	3.6e-4	1.6e-6	1.2e-6	PASS	PASS
Forge	2.8e-4	2.3e-4	9.5e-7	7.1e-7	PASS	PASS
Threshold	1e-3	1e-3	1e-5	1e-5	PASS	

Roofline Classification

Metric	Value
Kernel Arithmetic Intensity	~1.3 – 2.1 FLOPs/Byte
A100 Ridge Point	~153 FLOPs/Byte
Roofline Region	Slanted line (Memory Wall)
Implication	Further speedup requires reducing HBM reads/writes, not more FLOPs
Forge BW Utilisation	78.3% of peak — near optimal for a memory-bound kernel

Part B — Model-Level (Fused) Benchmarks



Model-Level = kernel swapped into an actual HuggingFace model during training. This is the real-world comparison, especially against Unsloth which fuses entire layers.

B1. Training Configuration

Model	Qwen3-8B
Task	SFT (Supervised Fine-Tuning)

Method	LoRA (rank=16, alpha=32)
Batch Size	4
Sequence Length	2048
Dtype	bf16
Optimizer	AdamW (lr=2e-4)
Gradient Checkpointing	Enabled
Steps Measured	100 training steps (after 10 warmup)

B2. Training Throughput & Efficiency

Metric	PyTorch (baseline)	Liger (patched)	Unsloth (fused)	Forge (patched)
Step Time — Mean (ms)	847.3	538.2	491.6	508.4
Step Time — Median (ms)	845.8	537.1	490.4	507.2
Step Time — P50 (ms)	845.8	537.1	490.4	507.2
Step Time — P95 (ms)	856.2	544.7	497.8	514.9
Step Time — P99 (ms)	861.4	548.3	501.2	518.1
Step Time — Std Dev (ms)	5.8	3.9	3.6	3.7
Throughput (tokens/sec)	9,667	15,221	16,662	16,106
Peak VRAM (GB)	68.4	42.3	35.1	38.9
Speedup vs PyTorch	1.00x	1.57x	1.72x	1.67x
VRAM Saved vs PyTorch	—	38.2%	48.7%	43.1%



Why Unsloth is slightly faster at model level: Unsloth fuses entire transformer layers (RMSNorm → Attention → MLP → Residual) into a single backward pass, eliminating intermediate tensor saves. Forge uses per-kernel autograd for modularity. This gap closes as we add more kernels and later implement layer-level manual autograd.

B3. Training Convergence Verification



Critical: Patched model loss must exactly match unpatched model loss. Any divergence means the kernel has a numerical bug.

Training Step	PyTorch Loss	Forge Loss	Δ Loss (absolute)	Verdict
Step 100	2.8473	2.8473	0.0000	Match
Step 250	2.3841	2.3841	0.0000	Match
Step 500	1.9228	1.9228	0.0000	Match
Step 750	1.6514	1.6514	0.0000	Match
Step 1000	1.4122	1.4122	0.0000	Match

Summary & Interpretation Template



Copy this section into your report and fill it in. This is the executive summary your teammates and reviewers will read first.

Kernel Report Card

Check	Status	Details
Forward correctness (bf16)	PASS	atol=2.8e-4 < 1e-3 threshold
Forward correctness (fp32)	PASS	atol=9.5e-7 < 1e-5 threshold
Backward gradcheck (fp64)	PASS	torch.autograd.gradcheck passed
Convergence (1000 steps)	PASS	0.0000 loss delta at all checkpoints
Isolation speedup vs PyTorch	2.40x	Fwd+Bwd at reference config
Model-level speedup vs PyTorch	1.67x	Qwen3-8B SFT training step time
VRAM reduction vs PyTorch	43.1%	Peak VRAM during training
Peak HW utilisation	78.3%	Memory BW utilisation (memory-bound kernel)
Scales linearly with input size	YES	Consistent speedup across all sweep dimensions
Forge vs Liger (isolation)	Faster	178.0 μ s vs 182.7 μ s (2.6% faster)
Forge vs Unsloth (isolation)	~Parity	178.0 μ s vs 175.2 μ s (1.6% slower)
Forge vs Unsloth (model-level)	Slightly Behind	508.4 ms vs 491.6 ms — expected due to layer-level fusion gap

Benchmark Submission Checklist



Before submitting your benchmark report, verify every item below.

- ☐ Test environment table filled (GPU, CUDA, PyTorch, Triton versions)
- ☐ Timing uses `torch.cuda.Event` — NOT `time.time()`
- ☐ L2 cache flushed between every run
- ☐ 3+ warmup runs discarded, 10+ timed runs recorded
- ☐ **Isolation benchmarks:** Forward, Backward, and Fwd+Bwd tables present
- ☐ **Scaling sweep:** seq_len, batch_size, and hidden_dim sweep tables present
- ☐ **Detailed stats:** Mean, Median, P50, P95, P99, Std Dev, Min, Max reported
- ☐ **Peak VRAM:** Measured and compared across all frameworks
- ☐ **Arithmetic Intensity:** Calculated and kernel classified as compute/memory bound
- ☐ **Peak Utilisation:** Compute % and Memory BW % reported; flagged if < 40%
- ☐ **Numerical Correctness:** atol, rtol for bf16 and fp32; gradcheck with fp64
- ☐ **Roofline classification:** Kernel placed on roofline model
- ☐ **Model-level benchmark:** Training step time, throughput, VRAM with Qwen3-8B
- ☐ **Model-level stats:** Mean, Median, P50, P95, P99, Std Dev for step time
- ☐ **Convergence test:** Loss curve matches unpatched model at 100, 250, 500, 750, 1000 steps
- ☐ **Kernel Report Card** summary table filled
- ☐ All numbers are **your actual measurements** — not copied from this template